

2520090240

Neha Sreekari K

Skill 2

1. CREATE TABLE

Used to create a new table with specified columns, data types, and constraints such as PRIMARY KEY.

```
mysql> USE sec_13;
Database changed
mysql> CREATE TABLE student_marks1 (
->     roll_no INT PRIMARY KEY,
->     name VARCHAR(50),
->     subject VARCHAR(50),
->     marks DECIMAL(5,2)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> SHOW TABLES;
+-----+
| Tables_in_sec_13 |
+-----+
| student          |
| student_marks1   |
+-----+
2 rows in set (0.00 sec)
```

2. INSERT

Used to add new records into a table.

```
mysql> -- 2. Insert Data
mysql> INSERT INTO student_marks1 (roll_no, name, subject, marks) VALUES
-> (1, 'Ravi', 'Math', 85.50),
-> (2, 'Sita', 'Math', 92.75),
-> (3, 'Anil', 'Math', 78.40),
-> (4, 'Priya', 'Math', 88.90),
-> (5, 'Vijay', 'Math', 80.25),
-> (6, 'subbusir', 'aws', 98.50),
-> (7, 'dwaraka', 'dbms', 95.75),
-> (8, 'ranjani', 'english', 97.40),
-> (9, 'kavithamam', 'aws1', 99.90),
-> (10, 'seetha', 'azure', 82.25);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM student_marks1;
+-----+-----+-----+-----+
| roll_no | name      | subject | marks |
+-----+-----+-----+-----+
| 1       | Ravi      | Math    | 85.50 |
| 2       | Sita      | Math    | 92.75 |
| 3       | Anil      | Math    | 78.40 |
| 4       | Priya     | Math    | 88.90 |
| 5       | Vijay     | Math    | 80.25 |
| 6       | subbusir  | aws     | 98.50 |
| 7       | dwaraka   | dbms    | 95.75 |
| 8       | ranjani   | english | 97.40 |
| 9       | kavithamam | aws1    | 99.90 |
| 10      | seetha    | azure   | 82.25 |
+-----+-----+-----+-----+
10 rows in set (0.00 sec)
```

3. SELECT

Used to retrieve and display data from a table.

```
mysql> SELECT COUNT(*) AS total_students
-> FROM student_marks1;
+-----+
| total_students |
+-----+
|          10 |
+-----+
1 row in set (0.01 sec)

mysql>
mysql> SELECT SUM(marks) AS total_marks
-> FROM student_marks1;
+-----+
| total_marks |
+-----+
|      899.60 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT AVG(marks) AS average_marks
-> FROM student_marks1;
+-----+
| average_marks |
+-----+
|    89.960000 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT MAX(marks) AS highest_marks
-> FROM student_marks1;
+-----+
| highest_marks |
+-----+
|        99.90 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT MIN(marks) AS lowest_marks
-> FROM student_marks1;
+-----+
| lowest_marks |
+-----+
|        78.40 |
+-----+
1 row in set (0.00 sec)

mysql> |
```

WHERE Clause

Used to filter records based on a condition. Operators used in the file include >, <, >=, BETWEEN, LIKE, and IN.

```
mysql> -- 5. Filtering with WHERE
mysql>
mysql> SELECT * FROM student_marks1 WHERE marks > 85;
+-----+-----+-----+-----+
| roll_no | name   | subject | marks |
+-----+-----+-----+-----+
| 1       | Ravi   | Math    | 85.50 |
| 2       | Sita   | Math    | 92.75 |
| 4       | Priya  | Math    | 88.90 |
| 6       | subbusir | aws     | 98.50 |
| 7       | dwaraka | dbms    | 95.75 |
| 8       | ranjani | english | 97.40 |
| 9       | kavithamam | aws1  | 99.90 |
+-----+-----+-----+-----+
7 rows in set (0.01 sec)

mysql>
mysql> SELECT * FROM student_marks1 WHERE marks >= 90;
+-----+-----+-----+-----+
| roll_no | name   | subject | marks |
+-----+-----+-----+-----+
| 2       | Sita   | Math    | 92.75 |
| 6       | subbusir | aws     | 98.50 |
| 7       | dwaraka | dbms    | 95.75 |
| 8       | ranjani | english | 97.40 |
| 9       | kavithamam | aws1  | 99.90 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM student_marks1 WHERE marks < 80;
+-----+-----+-----+-----+
| roll_no | name | subject | marks |
+-----+-----+-----+-----+
| 3       | Anil | Math    | 78.40 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT * FROM student_marks1 WHERE marks BETWEEN 80 AND 90;
+-----+-----+-----+-----+
| roll_no | name   | subject | marks |
+-----+-----+-----+-----+
| 1       | Ravi   | Math    | 85.50 |
| 4       | Priya  | Math    | 88.90 |
| 5       | Vijay  | Math    | 80.25 |
| 10      | seetha | azure   | 82.25 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM student_marks1 WHERE name LIKE 'P%';
+-----+-----+-----+-----+
| roll_no | name   | subject | marks |
+-----+-----+-----+-----+
| 4       | Priya  | Math    | 88.90 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT * FROM student_marks1 WHERE name IN ('Ravi', 'Sita', 'Vijay');
+-----+-----+-----+-----+
| roll_no | name   | subject | marks |
+-----+-----+-----+-----+
| 1       | Ravi   | Math    | 85.50 |
| 2       | Sita   | Math    | 92.75 |
| 5       | Vijay  | Math    | 80.25 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM student_marks1
-> WHERE marks > 85
-> AND (subject = 'Math' OR name LIKE 'P%');
+-----+-----+-----+-----+
| roll_no | name   | subject | marks |
+-----+-----+-----+-----+
| 1       | Ravi   | Math    | 85.50 |
| 2       | Sita   | Math    | 92.75 |
| 4       | Priya  | Math    | 88.90 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Aggregate Functions

Used to perform calculations on a group of records. Common functions are:

- COUNT() - counts records
- SUM() - calculates total
- AVG() - calculates average
- MAX() - finds maximum value
- MIN() - finds minimum value

```
mysql> -- 6. Update Queries
mysql>
mysql> UPDATE student_marks1
    -> SET marks = 90.00
    -> WHERE roll_no = 3;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE student_marks1
    -> SET subject = 'Remedial Math'
    -> WHERE marks < 80;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> SELECT * FROM student_marks1 WHERE roll_no = 3;
+-----+-----+-----+-----+
| roll_no | name | subject | marks |
+-----+-----+-----+-----+
|      3 | Anil | Math    | 90.00 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM student_marks1;
+-----+-----+-----+-----+
| roll_no | name      | subject | marks |
+-----+-----+-----+-----+
|      1 | Ravi      | Math    | 85.50 |
|      2 | Sita      | Math    | 92.75 |
|      3 | Anil      | Math    | 90.00 |
|      4 | Priya     | Math    | 88.90 |
|      5 | Vijay     | Math    | 80.25 |
|      6 | subbusir  | aws     | 98.50 |
|      7 | dwaraka   | dbms    | 95.75 |
|      8 | ranjani   | english | 97.40 |
|      9 | kavithamam | aws1    | 99.90 |
|     10 | seetha    | azure   | 82.25 |
+-----+-----+-----+-----+
10 rows in set (0.00 sec)
```

DELETE

Used to remove records from a table.

```
mysql> DELETE FROM student_marks1
-> WHERE marks < 75;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM student_marks1;
+-----+-----+-----+-----+
| roll_no | name      | subject | marks |
+-----+-----+-----+-----+
| 1       | Ravi      | Math    | 85.50 |
| 2       | Sita      | Math    | 92.75 |
| 3       | Anil      | Math    | 90.00 |
| 4       | Priya     | Math    | 88.90 |
| 6       | subbusir  | aws     | 98.50 |
| 7       | dwaraka   | dbms    | 95.75 |
| 8       | ranjani   | english | 97.40 |
| 9       | kavithamam | aws1    | 99.90 |
| 10      | seetha    | azure   | 82.25 |
+-----+-----+-----+-----+
9 rows in set (0.00 sec)

mysql> SELECT * FROM student_marks1 WHERE marks < 75;
Empty set (0.00 sec)
```

ORDER BY

Used to sort query results in ascending (ASC) or descending (DESC) order.

```
mysql> SELECT * FROM student_marks1 ORDER BY marks ASC;
+-----+-----+-----+-----+
| roll_no | name      | subject | marks |
+-----+-----+-----+-----+
| 10      | seetha    | azure   | 82.25 |
| 1       | Ravi      | Math    | 85.50 |
| 4       | Priya     | Math    | 88.90 |
| 3       | Anil      | Math    | 90.00 |
| 2       | Sita      | Math    | 92.75 |
| 7       | dwaraka   | dbms    | 95.75 |
| 8       | ranjani   | english | 97.40 |
| 6       | subbusir  | aws     | 98.50 |
| 9       | kavithamam | aws1    | 99.90 |
+-----+-----+-----+-----+
9 rows in set (0.00 sec)

mysql> SELECT * FROM student_marks1 ORDER BY marks DESC;
+-----+-----+-----+-----+
| roll_no | name      | subject | marks |
+-----+-----+-----+-----+
| 9       | kavithamam | aws1    | 99.90 |
| 6       | subbusir  | aws     | 98.50 |
| 8       | ranjani   | english | 97.40 |
| 7       | dwaraka   | dbms    | 95.75 |
| 2       | Sita      | Math    | 92.75 |
| 3       | Anil      | Math    | 90.00 |
| 4       | Priya     | Math    | 88.90 |
| 1       | Ravi      | Math    | 85.50 |
| 10      | seetha    | azure   | 82.25 |
+-----+-----+-----+-----+
9 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM student_marks1 ORDER BY name ASC;
```

roll_no	name	subject	marks
3	Anil	Math	90.00
7	dwaraka	dbms	95.75
9	kavithamam	aws1	99.90
4	Priya	Math	88.90
8	ranjani	english	97.40
1	Ravi	Math	85.50
10	seetha	azure	82.25
2	Sita	Math	92.75
6	subbusir	aws	98.50

```
9 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM student_marks1 ORDER BY name DESC;
```

roll_no	name	subject	marks
6	subbusir	aws	98.50
2	Sita	Math	92.75
10	seetha	azure	82.25
1	Ravi	Math	85.50
8	ranjani	english	97.40
4	Priya	Math	88.90
9	kavithamam	aws1	99.90
7	dwaraka	dbms	95.75
3	Anil	Math	90.00

```
9 rows in set (0.00 sec)
```

GROUP BY

Used to group records having the same value and perform aggregate calculations for each group.

```
mysql> SELECT subject, SUM(marks) AS total_marks  
-> FROM student_marks1  
-> GROUP BY subject;
```

subject	total_marks
Math	357.15
aws	98.50
dbms	95.75
english	97.40
aws1	99.90
azure	82.25

6 rows in set (0.00 sec)

```
mysql>  
mysql> SELECT subject, AVG(marks) AS avg_marks  
-> FROM student_marks1  
-> GROUP BY subject;
```

subject	avg_marks
Math	89.287500
aws	98.500000
dbms	95.750000
english	97.400000
aws1	99.900000
azure	82.250000

6 rows in set (0.00 sec)

```
mysql>  
mysql> SELECT subject, COUNT(*) AS num_students  
-> FROM student_marks1  
-> GROUP BY subject;
```

subject	num_students
Math	4
aws	1
dbms	1
english	1
aws1	1
azure	1

6 rows in set (0.00 sec)

HAVING

Used to filter grouped results based on aggregate conditions. It is commonly used with GROUP BY.

```
mysql> -- 10. Having
mysql>
mysql> SELECT subject, AVG(marks) AS avg_marks
-> FROM student_marks1
-> GROUP BY subject
-> HAVING AVG(marks) > 90;
```

subject	avg_marks
aws	98.500000
dbms	95.750000
english	97.400000
aws1	99.900000

4 rows in set (0.00 sec)

```
mysql>
mysql> SELECT subject, COUNT(*) AS num_students
-> FROM student_marks1
-> GROUP BY subject
-> HAVING COUNT(*) > 1;
```

subject	num_students
Math	4

1 row in set (0.00 sec)

